
ENTERPRISE TECHNICAL BLUEPRINT

SupportPilot

AI Ticket Resolution Agent — Development & Implementation Plan

DOCUMENT TYPE	Technical Design & Development Blueprint (implementation-ready)
PROJECT CODE	SP-ATR-001
DOMAIN	IT Service Management / Applied AI
DURATION	8 Weeks (4 Milestones) + Week 0 Mobilisation
ARCHITECTURE	React SPA • Django REST Core API • FastAPI AI Service • PostgreSQL + pgvector
AUDIENCE	Engineering Team, Tech Lead, QA, DevOps, Reviewers
STATUS	Baseline v1.0 — approved for build
NOTE	This document contains no source code. It is a specification a team implements from.

Table of Contents

1. Scope, Objectives & Success Criteria

What is being built and how success is measured

2. Solution Architecture Overview

Layered architecture and service decomposition

3. Recommended Technology Stack

Per-layer choices with rationale and alternatives

4. Frontend Blueprint

Folder structure, page inventory, routing, state, conventions

5. Backend Blueprint

Service split, folder structures, API architecture, endpoint catalogue

6. Database Design

Entities, columns, relationships, indexes, vector store

7. Authentication, Authorisation & Roles

Token strategy, RBAC matrix, security controls

8. AI / ML Module Design

Classification, RAG, multi-agent orchestration, evaluation

9. Third-Party Integrations

Jira, email, LLM provider, observability

10. End-to-End Project Workflow

Ticket lifecycle, state machine, escalation rules

11. Development Phases — Step by Step

Week 0 to Week 8, task-level plan with Definition of Done

12. Deployment Architecture

Environments, containers, CI/CD, scaling, observability

13. Non-Functional Requirements, Testing & Security

Performance, QA strategy, compliance

14. Team Structure, Ways of Working & Risk Register

Squads, Git flow, ceremonies, risks

1. Scope, Objectives & Success Criteria

1.1 Problem Restatement

IT support desks absorb a high volume of repetitive requests — VPN failures, password resets, software installation, printer and hardware faults, network connectivity. Each is individually cheap and collectively expensive. Engineers spend their time re-deriving answers that already exist in knowledge bases and SOP repositories. The measurable symptoms are long first-response times, inconsistent resolution quality, high cost per ticket, and poor employee satisfaction.

SupportPilot addresses this by inserting an AI layer between ticket intake and human engineers. Every incoming ticket is classified, enriched with retrieved knowledge, and given a candidate resolution. Tickets the system can confidently resolve are answered automatically; the rest are routed to the right team with the investigation already done.

1.2 In Scope

Capability	Description
Multi-channel intake	Web portal form, inbound email mailbox polling, and REST ingestion endpoint for external service desks.
AI classification	Category, sub-category, severity and priority assignment with a confidence score for every ticket.
Knowledge retrieval	Semantic search over enterprise KB articles, troubleshooting guides and SOPs using a vector store.
Resolution generation	Grounded, step-by-step troubleshooting output with citations back to source articles.
Multi-agent workflow	Coordinated Diagnosis, Retrieval, Resolution, Validation and Escalation agents for complex tickets.
Escalation & routing	Rule + confidence driven routing to department queues, with SLA timers and re-assignment.
Jira integration	Bi-directional issue creation, status sync and comment mirroring.
Email automation	Acknowledgement, resolution delivery, status change and escalation notification templates.
Analytics dashboard	Volume, resolution rate, AI deflection rate, MTTR, agent performance, CSAT.

1.3 Explicitly Out of Scope (Phase 1)

- Voice or telephony intake channels; chat-widget live conversation.
- Automated execution of remediation actions on endpoints (script running, account unlocking). The system *recommends*; humans *act*.
- Fine-tuning of a foundation model. Phase 1 uses prompt engineering, retrieval and a lightweight supervised classifier only.
- Multi-tenant SaaS isolation. Phase 1 is single-organisation.

1.4 Success Criteria (contractual, mapped to milestones)

Milestone	Criterion	Measurement Method	Target
M1 / Week 2	Ticket classification coverage	Labelled hold-out set of 500 historical tickets	≥ 90% correctly categorised
M1 / Week 2	Severity prediction accuracy	Same hold-out set, severity label	≥ 85%
M2 / Week 4	Retrieval relevance	Recall@5 on 100 curated ticket→article pairs	≥ 0.85
M2 / Week 4	Grounded responses with citations	Manual review of 100 generated resolutions	100% carry ≥ 1 source reference
M3 / Week 6	Multi-agent workflow completion	Automated integration suite over 50 complex tickets	≥ 95% complete without error
M3 / Week 6	Jira + email reliability	Integration test job, 200 events	≥ 99% delivered, zero duplicates
M4 / Week 8	Response generation latency	p95 measured at API gateway under 20 concurrent users	< 5 seconds
M4 / Week 8	User satisfaction	Post-resolution CSAT survey during UAT	≥ 85%

2. Solution Architecture Overview

2.1 Architectural Style

SupportPilot uses a **modular service architecture** — not a monolith, not fine-grained microservices. Three deployable units keep the AI workload isolated from transactional business logic while avoiding the operational overhead of a large service mesh. This is the right granularity for a team of this size and a delivery window of eight weeks.

Deployable Unit	Technology	Responsibility
Web Client	React + Vite SPA	All user interfaces: portal, agent console, knowledge manager, analytics, admin.
Core API	Django + DRF	Identity, RBAC, ticket CRUD and lifecycle, SLA, escalation rules, integrations, analytics aggregation, audit. Owns the system of record.
AI Service	FastAPI	Classification, embedding, retrieval, generation, agent orchestration. Stateless, horizontally scalable, no business rules.
Async Workers	Celery + Redis	Email polling, Jira sync, KB re-indexing, notification dispatch, analytics rollups, long-running agent runs.

Why Django for core and FastAPI for AI?

Django brings a mature ORM, migrations, admin, authentication and permissions out of the box — exactly the transactional plumbing an ITSM system needs, and none of which is worth rebuilding. FastAPI brings native async, which matters when a single request fans out to an embedding call, a vector query and two LLM calls. Splitting them also lets you scale AI pods on GPU/CPU-heavy nodes independently of the CRUD tier, and lets an AI failure degrade gracefully instead of taking the ticketing system down.

2.2 Layer Model

Layer	Components
Presentation	Support Portal (employee) • Agent Console (support staff) • Analytics Dashboard • Admin Console • Transactional email templates
API Gateway / Edge	Nginx ingress • TLS termination • rate limiting • request ID injection • WAF rules
Application (Core)	Auth & RBAC • Ticket lifecycle engine • SLA engine • Routing & escalation rules • Integration adapters • Analytics service • Audit
AI Intelligence	Classification engine • Severity model • RAG pipeline • Multi-agent orchestrator • Confidence scorer • Guardrails
Knowledge	KB article store • Chunk + embedding store (pgvector) • SOP repository • Document ingestion pipeline
Automation	Celery beat schedules • Jira adapter • SMTP/IMAP adapter • Notification dispatcher • Webhook receiver
Data	PostgreSQL (OLTP) • pgvector (embeddings) • Redis (cache, broker, rate limits) • Object storage (attachments) • Analytics rollup tables
Observability	Prometheus • Grafana • Loki • Sentry • Langfuse (LLM tracing) • structured JSON logs

2.3 Request Flow (happy path)

1. User submits ticket via portal / email / external desk → Core API validates, persists `tickets` row with status `NEW`, returns ticket ID immediately.

2. Core API enqueues `process_ticket` Celery task. The user is never made to wait for AI.

3. Worker calls AI Service `/v1/classify` → category, sub-category, severity, priority, confidence. Persisted to `ticket_classifications`.

4. Worker calls AI Service `/v1/resolve` → orchestrator runs Diagnosis → Retrieval → Resolution → Validation agents. Each step logged to `agent_steps`.

5. Resolution + citations + confidence returned, persisted to `ticket_responses`.

6. Decision gate: confidence $\geq$ auto-resolve threshold → send resolution email, status `AI_RESOLVED`, await user confirmation. Otherwise → create escalation, route to team queue, create Jira issue, notify assignee.

7. All state changes emit audit events and update analytics rollups; the dashboard reflects them in near real time via polling or SSE.

3. Recommended Technology Stack

3.1 Frontend

Concern	Choice	Rationale / Notes
Framework	React 18 + TypeScript	Strict mode on. TypeScript is mandatory — the ticket/classification/agent domain has enough shape that untyped code will rot.
Build tool	Vite 5	Fast HMR, simple env handling, first-class TS.
Styling	Tailwind CSS v4	Utility-first with a project design-token layer. No component CSS files.
Component library	shadcn/ui (Radix primitives)	Copy-in components, fully ownable, accessible by default. Avoids a heavyweight theme dependency.
Server state	TanStack Query v5	Caching, background refetch, optimistic updates, polling for live ticket status.
Client state	Zustand	Auth session, UI preferences, filter state. Deliberately small.
Routing	React Router v6 (data router)	Nested layouts, loaders, route-level role guards.
Forms	React Hook Form + Zod	Schema-first validation; Zod schemas double as TS types shared with API contracts.
Charts	Recharts	Sufficient for volume, trend, distribution and gauge visuals.
HTTP	Axios instance with interceptors	Token refresh, correlation ID, global error normalisation.
Realtime	SSE (primary) / polling fallback	Agent-run progress and dashboard tiles. Full WebSocket only if bidirectional need appears.
Testing	Vitest + React Testing Library + Playwright	Unit, component, and E2E respectively.

3.2 Backend

Concern	Choice	Rationale / Notes
Core framework	Django 5.x + Django REST Framework	ORM, migrations, admin, auth, permissions, signals. The system of record.
AI framework	FastAPI + Pydantic v2 + Uvicorn	Async I/O, automatic OpenAPI, strict request/response schemas.
Task queue	Celery 5 + Redis broker	Separate queues: ingest, ai, integrations, notifications, analytics.
Scheduler	Celery Beat (django-celery-beat)	Mailbox polling, Jira sync, KB reindex, SLA breach sweeps, nightly rollups.
API docs	drf-spectacular + FastAPI OpenAPI	Contract published to the frontend team on day one of each milestone.
Validation	DRF serializers / Pydantic models	No unvalidated dict passing across service boundaries.
Auth	SimpleJWT + optional OIDC/SSO	Access + refresh tokens; SSO via Azure AD/Okta as a Phase 1.5 switch.

3.3 Data, AI and Platform

Concern	Choice	Rationale / Notes
Primary database	PostgreSQL 16	Relational, transactional, JSONB for flexible AI payloads, full-text search as a hybrid-retrieval component.
Vector store	pgvector extension (HNSW index)	One database to operate and back up. Move to Qdrant only if the corpus exceeds a few million chunks.
Cache / broker	Redis 7	Celery broker, response cache, rate limiting, idempotency keys, distributed locks.
Object storage	S3-compatible (AWS S3 / MinIO)	Ticket attachments, KB source documents, exported reports. Pre-signed URLs only.
LLM provider	Provider-abstracted (Claude / GPT-class)	A single LLMPProvider interface. Never call a vendor SDK directly from business code.
Embeddings	Hosted embedding API or BGE-M3 self-hosted	Decide in Week 1 on cost/latency; the interface must make the choice swappable.
Classical ML	scikit-learn (+ optional DistilBERT)	TF-IDF/embedding + linear model baseline for classification. Cheap, fast, explainable, and a fallback when the LLM is unavailable.
Agent orchestration	LangGraph (or an explicit in-house state machine)	Graph-based control flow with checkpointing, retries and per-node tracing.
LLM observability	Langfuse (self-hosted)	Prompt versions, token cost, latency, per-trace inspection. Non-negotiable for debugging agents.
Containerisation	Docker + Docker Compose (dev) / Kubernetes (prod)	Same image promoted across environments.
CI/CD	GitHub Actions	Lint → test → build → scan → deploy, with environment gates.

Monitoring	Prometheus + Grafana + Loki + Sentry	Metrics, dashboards, log aggregation, error tracking.
Secrets	Vault / AWS Secrets Manager / sealed secrets	No credential ever committed; <code>.env.example</code> holds keys with empty values only.

4. Frontend Blueprint

4.1 Structural Conventions

The frontend follows a **feature-sliced** layout. A "page" is a thin composition layer: it arranges feature components and owns the route. All data fetching, domain types and business components live under `features/`. Nothing in `pages/` should contain an API call or a business rule. This is the single rule that keeps a codebase of this size navigable.

- Cross-feature imports go through a feature's `index.ts` barrel only.
- `components/ui/` holds dumb, domain-free primitives. If it mentions "ticket", it does not belong there.
- Every feature owns its API functions, query keys, hooks and types together, so a feature can be deleted in one operation.
- File naming: `PascalCase.tsx` for components, `camelCase.ts` for hooks/utils, `kebab-case` for folders.

4.2 Folder Structure

```
supportpilot-web/
├── public/
│   ├── favicon.svg
│   └── locales/                # i18n JSON (en, ta) if required
├── src/
│   ├── app/
│   │   ├── App.tsx            # root shell
│   │   ├── router.tsx         # route tree + role guards + lazy imports
│   │   ├── providers/
│   │   │   ├── QueryProvider.tsx
│   │   │   ├── AuthProvider.tsx
│   │   │   ├── ThemeProvider.tsx
│   │   │   └── ToastProvider.tsx
│   │   └── layouts/
│   │       ├── AuthLayout.tsx    # centred card, no chrome
│   │       ├── PortalLayout.tsx  # employee self-service shell
│   │       ├── ConsoleLayout.tsx # sidebar + topbar for support staff
│   │       └── AdminLayout.tsx
│   ├── pages/                 # route components only - no logic
│   │   ├── auth/
│   │   │   ├── LoginPage.tsx
│   │   │   ├── ForgotPasswordPage.tsx
│   │   │   └── ResetPasswordPage.tsx
│   │   ├── portal/
│   │   │   ├── MyTicketsPage.tsx
│   │   │   ├── NewTicketPage.tsx
│   │   │   ├── TicketTrackPage.tsx
│   │   │   └── SelfHelpSearchPage.tsx
│   │   ├── dashboard/
│   │   │   └── OverviewPage.tsx
│   │   ├── tickets/
│   │   │   ├── TicketListPage.tsx
│   │   │   ├── TicketDetailPage.tsx
│   │   │   ├── TicketQueuePage.tsx
│   │   │   └── BulkImportPage.tsx
│   │   ├── ai-agent/
│   │   │   ├── AgentWorkbenchPage.tsx
│   │   │   ├── AgentRunDetailPage.tsx
│   │   │   └── PromptPlaygroundPage.tsx
│   │   ├── knowledge/
│   │   │   ├── ArticleListPage.tsx
│   │   │   ├── ArticleEditorPage.tsx
│   │   │   ├── ArticleDetailPage.tsx
│   │   │   └── IngestionStatusPage.tsx
│   │   ├── escalations/
│   │   │   ├── EscalationBoardPage.tsx
│   │   │   └── SlaMonitorPage.tsx
│   │   ├── analytics/
│   │   │   ├── AnalyticsOverviewPage.tsx
│   │   │   ├── AgentPerformancePage.tsx
│   │   │   ├── AiQualityPage.tsx
│   │   │   └── ReportBuilderPage.tsx
│   │   ├── integrations/
│   │   │   ├── IntegrationsPage.tsx
│   │   │   ├── JiraConfigPage.tsx
│   │   │   └── EmailConfigPage.tsx
│   │   ├── admin/
│   │   │   ├── UsersPage.tsx
│   │   │   ├── RolesPage.tsx
│   │   │   ├── TeamsPage.tsx
│   │   │   ├── SlaPolicyPage.tsx
│   │   │   ├── RoutingRulesPage.tsx
│   │   │   ├── AiSettingsPage.tsx
│   │   │   └── AuditLogPage.tsx
│   │   └── errors/
│   │       ├── NotFoundPage.tsx
│   │       └── ForbiddenPage.tsx
│   └── features/
│       ├── auth/
│       │   ├── api/authApi.ts
│       │   ├── hooks/{useLogin,useSession,usePermissions}.ts
│       │   ├── components/{LoginForm,RoleGuard}.tsx
│       │   └── types.ts
```

```

├── index.ts
├── tickets/
│   ├── api/{ticketApi,ticketKeys}.ts
│   ├── hooks/{useTickets,useTicket,useCreateTicket,useTicketActions}.ts
│   ├── components/
│   │   ├── TicketTable.tsx
│   │   ├── TicketFilterBar.tsx
│   │   ├── TicketForm.tsx
│   │   ├── TicketTimeline.tsx
│   │   ├── TicketStatusBadge.tsx
│   │   ├── PriorityChip.tsx
│   │   ├── AttachmentUploader.tsx
│   │   └── CommentThread.tsx
│   ├── schemas/ticketSchema.ts          # Zod
│   ├── types.ts
│   └── index.ts
├── classification/
│   ├── api/classificationApi.ts
│   ├── components/{ClassificationPanel,ConfidenceMeter,
│   │               CategoryBreakdown,ReclassifyDialog}.tsx
│   └── index.ts
├── knowledge/
│   ├── api/knowledgeApi.ts
│   ├── components/{ArticleEditor,SemanticSearchBox,
│   │               RetrievedSourceCard,RelevanceBadge}.tsx
│   └── index.ts
├── resolutions/
│   ├── api/resolutionApi.ts
│   ├── components/{ResolutionPanel,CitationList,
│   │               StepChecklist,FeedbackWidget}.tsx
│   └── index.ts
├── agents/
│   ├── api/agentApi.ts
│   ├── hooks/useAgentRunStream.ts      # SSE subscription
│   ├── components/{AgentPipeline,AgentStepCard,
│   │               WorkflowActivityLog,TokenCostBadge}.tsx
│   └── index.ts
├── escalations/
├── analytics/
│   ├── api/analyticsApi.ts
│   ├── components/{KpiTile,VolumeTrendChart,ResolutionMixChart,
│   │               SlaGauge,AgentLeaderboard,CsatChart}.tsx
│   └── index.ts
├── integrations/
├── admin/
├── components/
│   ├── ui/                                # Button, Input, Select, Dialog, Tabs, Toast...
│   ├── common/                          # PageHeader, EmptyState, ErrorBoundary,
│   │                                       # LoadingSkeleton, ConfirmDialog, Breadcrumbs
│   ├── data/                            # DataTable, Pagination, ColumnFilter, ExportButton
│   └── charts/                          # ChartCard wrapper, tooltip theme
├── hooks/                                # useDebounce, usePagination, useMediaQuery,
│                                         # useLocalStorage, useInterval
├── lib/
│   ├── axios.ts                          # instance + interceptors + refresh queue
│   ├── queryClient.ts
│   ├── rbac.ts                          # can(user, action, resource)
│   ├── sse.ts
│   ├── formatters.ts                   # date, duration, currency, relative time
│   └── constants.ts                    # enums mirrored from backend
├── store/
│   ├── authStore.ts
│   ├── uiStore.ts
│   └── filterStore.ts
├── types/                                # api.d.ts (generated from OpenAPI), models.ts
├── config/                              # env.ts, routes.ts, navigation.ts, permissions.ts
├── styles/                              # globals.css, tokens.css
├── main.tsx
├── tests/
│   ├── unit/
│   ├── component/
│   └── e2e/                            # Playwright specs per user journey
├── .env.example
├── vite.config.ts
├── tailwind.config.ts
├── tsconfig.json
└── eslint.config.js

```

4.3 Page & Route Inventory

Route	Page	Allowed Roles	Purpose & Key Elements
/login	LoginPage	Public	Credential or SSO sign-in, MFA challenge slot.
/portal/tickets	MyTicketsPage	Employee	Own tickets, status chips, filter by state, quick reopen.
/portal/tickets/new	NewTicketPage	Employee	Submission form (title, description, department, attachments); shows suggested KB articles live as the user types.
/portal/tickets/:id	TicketTrackPage	Employee	Read-only timeline, AI resolution steps, "Did this solve it?" feedback, reopen action.

/portal/self-help	SelfHelpSearchPage	Employee	Semantic KB search before raising a ticket — the cheapest deflection channel.
/dashboard	OverviewPage	Agent, Engineer, Lead, Admin	KPI tiles (resolved, avg resolution time, AI resolution rate, CSAT), volume trend, escalation funnel.
/tickets	TicketListPage	Agent, Engineer, Lead, Admin	Master table: server-side pagination, saved views, multi-facet filters, bulk actions.
/tickets/queue	TicketQueuePage	Agent, Engineer	Team/personal work queue ordered by SLA risk; claim/assign actions.
/tickets/:id	TicketDetailPage	Agent, Engineer, Lead, Admin	Three-pane workspace: ticket + comments, AI classification panel, AI resolution with citations. Actions: accept, edit & send, reclassify, escalate, resolve, close.
/ai-agent	AgentWorkbenchPage	Engineer, Lead, Admin	Live view of agent runs, per-agent status, throughput, failure list.
/ai-agent/runs/:id	AgentRunDetailPage	Engineer, Lead, Admin	Step-by-step trace: inputs, retrieved chunks, prompts used, tokens, latency, confidence.
/ai-agent/playground	PromptPlaygroundPage	Admin	Test a prompt version against a sample ticket without affecting production.
/knowledge	ArticleListPage	Knowledge Mgr, Admin	Article inventory with index status, version, last-embedded timestamp.
/knowledge/new, /knowledge/:id/edit	ArticleEditorPage	Knowledge Mgr, Admin	Markdown editor, category/tags, publish workflow, "re-embed on save".
/knowledge/ingestion	IngestionStatusPage	Knowledge Mgr, Admin	Bulk document upload, chunking progress, embedding failures, re-index trigger.
/escalations	EscalationBoardPage	Engineer, Lead, Admin	Kanban by team; drag to reassign; escalation reason and age.
/escalations/sla	SlaMonitorPage	Lead, Admin	At-risk and breached tickets, time remaining, breach history.
/analytics	AnalyticsOverviewPage	Lead, Admin	Volume, category mix, deflection rate, MTTR trend, period comparison.
/analytics/agents	AgentPerformancePage	Lead, Admin	Human agent leaderboard: assigned, resolved, avg handling time, CSAT.
/analytics/ai-quality	AiQualityPage	Lead, Admin	Classification accuracy over time, retrieval relevance, hallucination flags, override rate, token cost.
/analytics/reports	ReportBuilderPage	Lead, Admin	Custom date range and dimensions; CSV/PDF export; scheduled email delivery.
/integrations	IntegrationsPage	Admin	Connection cards with health, last sync, error count.
/integrations/jira	JiraConfigPage	Admin	Project key, issue type, field mapping, status mapping, sync direction, test connection.
/integrations/email	EmailConfigPage	Admin	IMAP/SMTP settings, polling interval, template editor with variable preview.
/admin/users	UsersPage	Admin	Create/deactivate users, assign roles, department, team membership.
/admin/roles	RolesPage	Admin	Role definitions and permission toggles.
/admin/sla	SlaPolicyPage	Admin	Response/resolution targets per priority and business calendar.
/admin/routing	RoutingRulesPage	Admin	Rule builder: category/severity → team, with ordering and enable toggle.
/admin/ai-settings	AiSettingsPage	Admin	Auto-resolve confidence threshold, model selection, max tokens, retrieval top-k, guardrail toggles.
/admin/audit	AuditLogPage	Admin	Immutable activity log with actor, action, target, before/after, filterable.

4.4 State Management Policy		
Kind of state	Owner	Rule
Server data (tickets, articles, analytics)	TanStack Query	Never mirrored into Zustand. Query keys are centralised per feature (<code>ticketKeys.list(filters)</code>).
Session & permissions	Zustand <code>authStore</code>	Access token in memory; refresh token in <code>httpOnly</code> cookie. Never in <code>localStorage</code> .
UI preferences (theme, sidebar,	Zustand <code>uiStore</code> + <code>localStorage</code>	Non-sensitive only.

density)		
Filters & pagination	URL search params	Filters must be shareable and back-button safe. filterStore only holds unsaved draft state.
Form state	React Hook Form	Local to the form; Zod resolver mandatory.
Streaming agent progress	SSE hook + query cache patch	Stream updates patch the existing query cache rather than triggering refetch storms.

5. Backend Blueprint

5.1 Core API — Folder Structure (Django + DRF)

Each Django app follows the same internal contract: **models** hold persistence, **selectors** hold read queries, **services** hold write/business operations, **views** stay thin. Views never contain business logic and never write to more than one model directly — they delegate to a service function. This convention is what makes the codebase reviewable by a large team.

```
supportpilot-core/
├── config/
│   ├── settings/
│   │   ├── base.py                # shared settings, installed apps, DRF, logging
│   │   ├── local.py
│   │   ├── staging.py
│   │   └── production.py
│   ├── urls.py                   # /api/v1/ mount points
│   ├── celery.py
│   ├── asgi.py
│   └── wsgi.py
├── apps/
│   ├── accounts/
│   │   ├── models.py             # User, Role, Department, Team, TeamMembership
│   │   ├── serializers.py
│   │   ├── views.py              # login, refresh, me, users, roles
│   │   ├── services.py           # create_user, assign_role, deactivate
│   │   ├── selectors.py
│   │   ├── permissions.py        # IsAdmin, IsSupportStaff, IsTicketOwner
│   │   ├── urls.py
│   │   └── tests/
│   ├── tickets/
│   │   ├── models.py            # Ticket, TicketComment, TicketAttachment,
│   │   │                       # TicketStatusHistory, SlaPolicy, SlaTracker
│   │   ├── serializers.py
│   │   ├── views.py
│   │   ├── services.py           # create_ticket, transition_status, assign,
│   │   │                       # reopen, close, apply_sla
│   │   ├── selectors.py         # queue queries, filtered lists, sla_at_risk
│   │   ├── state_machine.py     # allowed transitions, guards
│   │   ├── filters.py
│   │   ├── tasks.py             # process_ticket, sla_breach_sweep
│   │   ├── signals.py
│   │   └── tests/
│   ├── classification/
│   │   ├── models.py            # TicketClassification, CategoryTaxonomy
│   │   ├── services.py          # request_classification, apply_result,
│   │   │                       # manual_override
│   │   ├── views.py             # reclassify, taxonomy CRUD
│   │   └── tests/
│   ├── knowledge/
│   │   ├── models.py            # KnowledgeArticle, ArticleVersion,
│   │   │                       # ArticleChunk, IngestionJob
│   │   ├── services.py          # publish_article, trigger_reindex
│   │   ├── views.py             # article CRUD, search proxy, ingestion
│   │   ├── tasks.py             # ingest_document, reembed_article
│   │   └── tests/
│   ├── resolutions/
│   │   ├── models.py           # TicketResponse, ResponseCitation,
│   │   │                       # ResolutionFeedback
│   │   ├── services.py          # store_response, accept, edit_and_send
│   │   ├── views.py
│   │   └── tests/
│   ├── agents/
│   │   ├── models.py            # AgentRun, AgentStep, AgentConfig
│   │   ├── services.py          # start_run, record_step, finalize_run
│   │   ├── views.py             # run_list/detail, SSE stream, retry
│   │   ├── tasks.py             # execute_agent_run
│   │   └── tests/
│   ├── escalations/
│   │   ├── models.py           # Escalation, RoutingRule, EscalationPolicy
│   │   ├── services.py          # evaluate_routing, escalate, reassign
│   │   ├── views.py
│   │   └── tests/
│   ├── integrations/
│   │   ├── jira/
│   │   │   ├── client.py        # REST wrapper, retry, rate-limit aware
│   │   │   ├── mapper.py        # ticket <-> issue field mapping
│   │   │   ├── tasks.py         # push_issue, pull_status, sync_comments
│   │   │   ├── webhooks.py      # inbound event receiver + signature check
│   │   │   └── models.py        # JiraLink, JiraSyncLog
│   │   ├── email/
│   │   │   ├── imap_poller.py
│   │   │   ├── smtp_sender.py
│   │   │   ├── parser.py        # thread-id, quoted-reply stripping
│   │   │   ├── templates/
│   │   │   ├── tasks.py
│   │   │   └── models.py        # EmailLog, EmailTemplate
│   │   └── models.py            # IntegrationConfig, IntegrationHealth
│   ├── notifications/
│   │   ├── models.py           # Notification, NotificationPreference
│   │   ├── dispatcher.py        # channel fan-out
│   │   ├── tasks.py
│   │   └── views.py
│   └── analytics/
```

```

├── models.py                # DailyTicketRollup, AgentPerformanceRollup,
                             # AiQualityRollup
├── aggregators.py
├── views.py                 # kpi, trends, leaderboard, ai-quality
├── exporters.py             # CSV / PDF report generation
├── tasks.py                 # nightly_rollup
├── audit/
│   ├── models.py           # ActivityLog (append-only)
│   ├── middleware.py        # actor + request-id capture
│   └── views.py
├── core/
│   ├── permissions.py       # base RBAC classes, object-level checks
│   ├── pagination.py        # standard cursor + page-number classes
│   ├── exceptions.py        # custom handler -> uniform error envelope
│   ├── responses.py         # success envelope helper
│   ├── middleware.py        # request ID, timing, tenant context
│   ├── throttling.py
│   ├── validators.py
│   └── enums.py             # single source of truth for status/priority
├── services/
│   ├── ai_client.py         # typed HTTP client for the FastAPI service
│   ├── storage.py           # S3 pre-signed URL helpers
│   └── cache.py
├── requirements/{base,local,production}.txt
├── docker/{Dockerfile,entrypoint.sh}
├── manage.py
└── pytest.ini

```

5.2 AI Service — Folder Structure (FastAPI)

```

supportpilot-ai/
├── app/
│   ├── main.py                # app factory, middleware, exception handlers
│   └── api/
│       └── v1/
│           ├── router.py
│           └── routes/
│               ├── health.py
│               ├── classify.py
│               ├── retrieve.py
│               ├── resolve.py
│               ├── agents.py
│               └── embed.py
│
│   ├── core/
│       ├── config.py          # pydantic-settings
│       ├── security.py        # service-to-service API key / mTLS
│       ├── logging.py         # structured JSON + trace id
│       └── errors.py
│
│   ├── llm/
│       ├── base.py            # LLMProvider protocol
│       ├── claude_provider.py
│       ├── openai_provider.py
│       ├── router.py          # model selection by task + cost tier
│       ├── token_guard.py      # budget enforcement, truncation policy
│       └── cache.py           # semantic response cache
│
│   ├── prompts/
│       ├── registry.py        # versioned prompt loader
│       ├── classification.v3.yaml
│       ├── diagnosis.v2.yaml
│       ├── resolution.v4.yaml
│       ├── validation.v1.yaml
│       └── email_draft.v2.yaml
│
│   ├── rag/
│       ├── ingestion.py       # loaders: md, pdf, docx, html, confluence
│       ├── chunking.py        # heading-aware + overlap strategy
│       ├── embedding.py       # batch embed, retry, dimension guard
│       ├── vectorstore.py     # pgvector adapter (swappable interface)
│       ├── retriever.py       # hybrid: dense + BM25/full-text
│       ├── reranker.py        # cross-encoder rerank of top-N
│       └── pipeline.py        # query -> expand -> retrieve -> rerank -> pack
│
│   ├── agents/
│       ├── base.py            # Agent protocol: run(state) -> state
│       ├── orchestrator.py    # graph definition, routing, checkpointing
│       ├── state.py           # typed shared state object
│       ├── diagnosis_agent.py
│       ├── retrieval_agent.py
│       ├── resolution_agent.py
│       ├── validation_agent.py
│       ├── escalation_agent.py
│       └── tools/
│           ├── kb_search.py
│           ├── ticket_history.py
│           └── similar_tickets.py
│
│   ├── ml/
│       ├── classifier/
│           ├── features.py
│           ├── train.py
│           ├── predict.py
│           └── labels.py
│       ├── severity/
│           ├── train.py
│           └── predict.py
│       ├── registry.py        # model versioning + load by tag
│       └── artifacts/         # .pkl / .onnx (gitignored, pulled from S3)
│
│   ├── guardrails/
│       ├── pii_scrubber.py    # mask before sending to provider
│       ├── injection_filter.py # prompt-injection heuristics on ticket text
│       ├── output_validator.py # schema + citation presence + refusal check
│       └── toxicity.py
│
│   ├── schemas/              # Pydantic request/response contracts
│   ├── evaluation/
│       ├── datasets/         # golden sets (versioned, in git)
│       ├── metrics.py        # accuracy, recall@k, MRR, faithfulness
│       ├── run_eval.py       # CI-invokable regression harness
│       └── reports/
│   ├── observability/
│       ├── tracing.py        # Langfuse spans per agent step
│       └── metrics.py        # Prometheus counters/histograms
│
├── tests/{unit,integration,eval}/
├── notebooks/               # exploration only, never imported
├── requirements.txt
└── Dockerfile

```

5.3 API Architecture & Conventions

Concern	Standard
Base path	/api/v1/ on Core; /v1/ on AI Service (internal network only, never public).

Versioning	URI-based major version. Breaking changes require a new version; additive fields do not.
Success envelope	{ success, data, meta } where meta carries pagination and request id.
Error envelope	{ success:false, error:{ code, message, details[], request_id } }. code is a stable machine-readable string such as TICKET_INVALID_TRANSITION.
Status codes	200 read, 201 create, 202 accepted (async AI work), 204 delete, 400 validation, 401 unauthenticated, 403 unauthorised, 404, 409 conflict, 422 semantic, 429 throttled, 5xx server.
Pagination	Cursor pagination for high-volume list endpoints (tickets, audit); page-number for admin lists. Default 25, max 100.
Filtering	Query params via django-filter: ?status=OPEN&priority=P1&category=NETWORK&assigned_to=me&created_after=.
Sorting	?ordering=-created_at, whitelisted fields only.
Idempotency	Idempotency-Key header required on ticket creation and all outbound integration pushes; keys stored in Redis for 24h.
Correlation	X-Request-ID generated at edge, propagated through Core → Celery → AI Service → LLM trace. Every log line carries it.
Rate limits	Anonymous 20/min, authenticated 120/min, ticket creation 10/min/user, AI endpoints 30/min/service-token.
Long-running work	Returns 202 with a run_id; client polls /agents/runs/{id} or subscribes to the SSE stream.
Service-to-service auth	Core → AI uses a signed service token plus network policy. The AI Service is never reachable from the internet.

5.3.1 Core API Endpoint Catalogue

Method	Endpoint	Roles	Description
POST	/auth/login	Public	Issue access + refresh tokens.
POST	/auth/refresh	Public (valid refresh)	Rotate access token.
POST	/auth/logout	Any	Blacklist refresh token.
GET	/auth/me	Any	Profile, roles, effective permissions.
POST	/tickets	Employee+	Create ticket; returns 201 and triggers async AI pipeline.
GET	/tickets	Scoped by role	List with filters; employees see only their own.
GET	/tickets/{id}	Owner, Support+	Full detail with classification, response, timeline.
PATCH	/tickets/{id}	Support+	Update mutable fields.
POST	/tickets/{id}/transition	Support+	State-machine-guarded status change.
POST	/tickets/{id}/assign	Lead, Admin	Assign to user or team.
POST	/tickets/{id}/comments	Owner, Support+	Add comment (internal or public flag).
POST	/tickets/{id}/attachments	Owner, Support+	Request pre-signed upload URL.
POST	/tickets/{id}/reopen	Owner	Reopen within the configured window.
POST	/tickets/ingest	Service token	External service-desk ingestion endpoint.
POST	/tickets/{id}/classify	Support+	Force re-classification.
PATCH	/classifications/{id}	Support+	Manual override; recorded as training signal.
GET	/taxonomy/categories	Any	Category/sub-category tree.
GET	/knowledge/articles	Any	List/search published articles.
POST	/knowledge/articles	Knowledge Mgr+	Create draft.
PUT	/knowledge/articles/{id}	Knowledge Mgr+	Update; creates a new version and queues re-embedding.
POST	/knowledge/articles/{id}/publish	Knowledge Mgr+	Publish and index.
POST	/knowledge/search	Any	Semantic + keyword hybrid search (proxies AI Service).
POST	/knowledge/ingest	Knowledge Mgr+	Bulk document ingestion job.
GET	/knowledge/ingest/{job_id}	Knowledge Mgr+	Ingestion progress and failures.
GET	/tickets/{id}/responses	Owner, Support+	Generated resolutions with citations.
POST	/responses/{id}/accept	Support+	Approve and dispatch to requester.
POST	/responses/{id}/edit-send	Support+	Human-edited send; diff stored for quality analysis.
POST	/responses/{id}/feedback	Employee	Helpful / not helpful + CSAT rating.
GET	/agents/runs	Engineer+	List agent runs with status and duration.
GET	/agents/runs/{id}	Engineer+	Full step trace.
GET	/agents/runs/{id}/stream	Engineer+	SSE progress stream.
POST	/agents/runs/{id}/retry	Engineer+	Re-run from last successful checkpoint.

GET	/escalations	Engineer+	Board data grouped by team/status.
POST	/escalations	Support+	Manual escalation with reason.
POST	/escalations/{id}/reassign	Lead, Admin	Move between teams.
GET/PUT	/admin/routing-rules	Admin	Ordered routing rule set.
GET/PUT	/admin/sla-policies	Admin	SLA targets per priority.
GET/PUT	/admin/ai-settings	Admin	Thresholds, model, top-k, guardrail flags.
GET/PUT	/integrations/jira	Admin	Config and field mapping.
POST	/integrations/jira/test	Admin	Connection test.
POST	/integrations/jira/webhook	Signed	Inbound Jira event receiver.
GET/PUT	/integrations/email	Admin	IMAP/SMTP config and templates.
GET	/analytics/kpi	Lead, Admin	Headline tiles for a date range.
GET	/analytics/trends	Lead, Admin	Time series by dimension.
GET	/analytics/ai-quality	Lead, Admin	Accuracy, override rate, latency, token cost.
POST	/analytics/reports/export	Lead, Admin	Queue CSV/PDF export job.
GET	/audit/logs	Admin	Filterable append-only activity log.
GET	/health, /ready	Public	Liveness and readiness probes.

5.3.2 AI Service Endpoint Catalogue (internal)

Method	Endpoint	Input	Output
POST	/v1/classify	ticket_id, subject, description, department	category, sub_category, severity, priority, confidence, model_version, latency_ms
POST	/v1/retrieve	query, top_k, filters, rerank flag	ranked chunks with article_id, title, snippet, relevance score
POST	/v1/resolve	ticket payload, classification, history	resolution steps, summary, citations[], confidence, escalation_recommended
POST	/v1/agents/run	ticket payload, workflow name	run_id (202 accepted)
GET	/v1/agents/run/{id}	—	status, steps[], final state, token usage
POST	/v1/embed	texts[], model	vectors[], dimension, model_version
POST	/v1/index/upsert	article_id, chunks[]	indexed count, failures[]
POST	/v1/draft-email	ticket, resolution, template, tone	subject, body_html, body_text
GET	/v1/models	—	active model versions and prompt versions

6. Database Design

6.1 Design Principles

- UUID primary keys on all business tables. Sequential integers leak volume and complicate any future data merge.
- `created_at`, `updated_at`, `created_by`, `updated_by` on every mutable table via an abstract base model.
- Soft delete (`is_deleted`, `deleted_at`) on tickets, articles and users. Hard delete only via a retention job.
- Enumerations stored as short uppercase strings with database `CHECK` constraints, mirrored in `core/enums.py` and the frontend `constants.ts`.
- All money-free numeric scores stored as `NUMERIC(5,4)` for confidence and relevance — never float, so comparisons are stable.
- AI payloads (raw model output, prompt snapshot, token counts) stored as `JSONB` so schema changes in the AI layer do not require migrations.

6.2 Core Entities

users

Column	Type	Notes
id	UUID PK	
email	VARCHAR(255) UNIQUE	Login identifier; indexed.
full_name	VARCHAR(150)	
password_hash	VARCHAR(255)	Argon2id. Null when SSO-only.
role	VARCHAR(30)	EMPLOYEE / AGENT / ENGINEER / LEAD / KNOWLEDGE_MGR / ADMIN
department_id	UUID FK → departments	Nullable.
is_active	BOOLEAN	Default true.
last_login_at	TIMESTAMPTZ	
created_at / updated_at	TIMESTAMPTZ	

tickets

Column	Type	Notes
id	UUID PK	
ticket_number	VARCHAR(20) UNIQUE	Human-facing, e.g. IT-2026-004521.
requester_id	UUID FK → users	Indexed.
assigned_to_id	UUID FK → users NULL	Indexed.
assigned_team_id	UUID FK → teams NULL	
subject	VARCHAR(255)	
description	TEXT	Full-text indexed (tsvector generated column).
channel	VARCHAR(20)	PORTAL / EMAIL / API / SERVICE_DESK
category	VARCHAR(60)	Set by AI, overridable.
sub_category	VARCHAR(60)	
severity	VARCHAR(10)	LOW / MEDIUM / HIGH / CRITICAL
priority	VARCHAR(4)	P1-P4
status	VARCHAR(25)	See state machine, section 10.2.
resolution_mode	VARCHAR(15)	AI_AUTO / AI_ASSISTED / MANUAL
sla_due_at	TIMESTAMPTZ	Computed at classification time.
first_response_at	TIMESTAMPTZ	For SLA reporting.
resolved_at / closed_at	TIMESTAMPTZ	
reopen_count	SMALLINT	Quality signal.
metadata	JSONB	Channel-specific extras (email headers, source system id).

Indexes: `(status, priority, sla_due_at)` for queue ordering; `(requester_id, created_at DESC)`; `(assigned_to_id, status)`; GIN on the description tsvector; partial index on `status IN ('NEW', 'IN_PROGRESS')` for the hot working set.

Remaining tables

Table	Key columns
departments	id, name, code, manager_id
teams	id, name, department_id, queue_key, is_escalation_target
team_memberships	id, team_id, user_id, is_lead
ticket_comments	id, ticket_id, author_id, body, visibility (PUBLIC/INTERNAL), source (HUMAN/AI/EMAIL), created_at
ticket_attachments	id, ticket_id, file_key, file_name, mime_type, size_bytes, uploaded_by, virus_scan_status
ticket_status_history	id, ticket_id, from_status, to_status, actor_id, reason, changed_at
ticket_classifications	id, ticket_id, category, sub_category, severity, priority, confidence NUMERIC(5,4), model_version, prompt_version, is_override, overridden_by, raw_output JSONB, created_at
ticket_responses	id, ticket_id, agent_run_id, response_text, response_html, confidence, model_version, token_input, token_output, latency_ms, status (DRAFT/SENT/REJECTED), sent_at
response_citations	id, response_id, article_id, chunk_id, relevance NUMERIC(5,4), snippet
resolution_feedback	id, response_id, ticket_id, user_id, was_helpful BOOLEAN, csat SMALLINT (1-5), comment, created_at
knowledge_articles	id, title, slug, category, tags TEXT[], content, status (DRAFT/PUBLISHED/ARCHIVED), version, author_id, source_url, last_indexed_at
article_versions	id, article_id, version, content, changed_by, changed_at
article_chunks	id, article_id, chunk_index, content, token_count, embedding VECTOR(1024), heading_path, created_at
ingestion_jobs	id, source_type, file_key, status, total_docs, processed, failed, error_log JSONB, started_at, finished_at
agent_runs	id, ticket_id, workflow, status (QUEUED/RUNNING/SUCCEEDED/FAILED/PARTIAL), started_at, finished_at, duration_ms, total_tokens, total_cost, error JSONB
agent_steps	id, run_id, step_index, agent_name, input JSONB, output JSONB, prompt_version, model, tokens_in, tokens_out, latency_ms, status, error
escalations	id, ticket_id, from_team_id, to_team_id, reason_code, reason_text, escalated_by (USER/SYSTEM), confidence_at_escalation, status, created_at, resolved_at
routing_rules	id, order_index, condition JSONB (category/severity/keyword), target_team_id, is_active
sla_policies	id, priority, first_response_minutes, resolution_minutes, business_calendar, is_active
jira_links	id, ticket_id, jira_issue_key, jira_status, jira_url, last_synced_at, sync_direction
jira_sync_logs	id, ticket_id, direction, payload JSONB, http_status, error, created_at
email_logs	id, ticket_id, direction (IN/OUT), template_key, to_address, subject, message_id, thread_id, status, provider_response JSONB, sent_at
email_templates	id, key, subject_template, body_template, variables JSONB, is_active
notifications	id, user_id, type, title, body, link, is_read, created_at
activity_logs	id, actor_id, action, entity_type, entity_id, before JSONB, after JSONB, ip, user_agent, request_id, created_at (append-only, monthly partitions)
daily_ticket_rollups	date, category, priority, team_id, received, resolved, ai_resolved, escalated, avg_resolution_minutes, avg_first_response_minutes
ai_quality_rollups	date, classification_accuracy, override_rate, avg_confidence, avg_retrieval_relevance, avg_latency_ms, total_tokens, total_cost

6.3 Vector Store Configuration

Parameter	Value / Policy
Extension	pgvector on the same PostgreSQL instance.
Dimension	1024 (adjust to the chosen embedding model; enforce with a check on insert).
Index	HNSW with cosine distance, m=16, ef_construction=64, tuned ef_search at query time.
Chunking	Heading-aware split, target 500-800 tokens, 15% overlap; the heading path is prepended to each chunk so retrieved text carries context.
Hybrid retrieval	Dense vector top-30 UNION PostgreSQL full-text top-30, fused with Reciprocal Rank Fusion, then cross-encoder rerank to top-5.
Re-index policy	On article publish, on model version change (full rebuild), and a weekly consistency sweep that finds chunks whose last_indexed_at predates the article's updated_at.
Staleness guard	Every retrieved chunk carries its article's updated_at; the resolution agent is instructed to flag guidance older than the configured freshness window.

7. Authentication, Authorisation & Roles

7.1 Authentication Strategy

Aspect	Decision
Primary mechanism	JWT access token (15 min) + refresh token (7 days, rotating, single-use).
Token storage	Access token in memory only. Refresh token in an httpOnly, Secure, SameSite=Strict cookie. No tokens in localStorage — that is an XSS-to-account-takeover path.
Rotation & reuse detection	Each refresh issues a new refresh token and blacklists the old one. Reuse of a blacklisted token invalidates the entire family and forces re-login.
Password policy	Argon2id hashing, minimum 12 characters, breach-list check, lockout after 5 failures in 15 minutes with exponential backoff.
Enterprise SSO	OIDC (Azure AD / Okta) as an alternative provider; role mapped from an IdP group claim. Designed for from day one, enabled by configuration.
MFA	TOTP, mandatory for ADMIN and LEAD roles.
Service accounts	Long-lived signed service tokens scoped to specific endpoints, used by the ingestion API and Core→AI calls. Rotated quarterly.
Session controls	Active session listing, remote revoke, forced logout on role change or deactivation.

7.2 Role Definitions

Role	Description
EMPLOYEE	End user. Raises tickets, tracks own tickets, searches self-help, gives feedback.
AGENT (L1)	First-line support. Works the queue, accepts or edits AI resolutions, escalates.
ENGINEER (L2/L3)	Specialist. Handles escalated tickets, inspects agent runs, retries workflows.
LEAD	Team manager. Assignment, SLA oversight, analytics, team configuration.
KNOWLEDGE_MGR	Owns the knowledge base: authoring, publishing, ingestion, index health.
ADMIN	Full system configuration: users, roles, routing, SLA, integrations, AI settings, audit.
SERVICE	Machine identity for ingestion and inter-service calls. No UI access.

7.3 RBAC Permission Matrix

Capability	EMP	AGENT	ENGR	LEAD	KM	ADMIN
Create ticket	Yes	Yes	Yes	Yes	Yes	Yes
View own tickets	Yes	Yes	Yes	Yes	Yes	Yes
View all tickets	–	Team	Team	Dept	–	All
Assign / reassign	–	Self-claim	Self-claim	Yes	–	Yes
Change status	Reopen only	Yes	Yes	Yes	–	Yes
Accept / send AI resolution	–	Yes	Yes	Yes	–	Yes
Override classification	–	Yes	Yes	Yes	–	Yes
Escalate	–	Yes	Yes	Yes	–	Yes
View agent run traces	–	Summary	Full	Full	–	Full
Retry agent run	–	–	Yes	Yes	–	Yes
Author / publish KB article	–	Draft	Draft	Draft	Yes	Yes
Run bulk ingestion	–	–	–	–	Yes	Yes
View analytics	–	Own	Own	Team	KB only	All
Export reports	–	–	–	Yes	–	Yes
Configure routing / SLA	–	–	–	Propose	–	Yes
Configure integrations	–	–	–	–	–	Yes
Change AI thresholds / model	–	–	–	–	–	Yes
Manage users & roles	–	–	–	–	–	Yes
View audit log	–	–	–	–	–	Yes

Enforcement rule

Role checks are enforced in three places, and all three are mandatory: route guards in the React router (UX only), DRF permission classes at the view (coarse), and object-level checks inside service functions (authoritative). Frontend guards are never treated as security. Every list endpoint applies a role-scoped queryset filter, not a post-fetch filter.

8. AI / ML Module Design

8.1 Module Map

Module	Milestone	Approach	Primary Metric
Ticket Classification	M1	Hybrid: supervised classifier (fast path) + LLM (fallback/low-confidence path)	Category accuracy $\geq 90\%$
Severity & Priority Scoring	M1	Gradient-boosted classifier over text embeddings + structured features	Severity accuracy $\geq 85\%$
Knowledge Retrieval (RAG)	M2	Hybrid dense + lexical retrieval with cross-encoder rerank	Recall@5 ≥ 0.85
Resolution Generation	M2	Grounded generation with mandatory citations and refusal on insufficient context	Faithfulness ≥ 0.9 ; 100% cited
Multi-Agent Orchestration	M3	Directed graph with typed shared state, checkpointing, per-node retry	Run completion $\geq 95\%$
Confidence & Validation	M3	Composite score: retrieval relevance, self-consistency, validator agent verdict	Calibration error < 0.1
Email Drafting	M3	Template-constrained generation with tone control	Human edit rate $< 30\%$
Analytics & Anomaly Detection	M4	Rollup aggregation + simple statistical anomaly flags on volume/category	Dashboard freshness < 5 min

8.2 Module 1 — Ticket Classification Engine

Design decision: do not send every ticket to an LLM. Most tickets are near-duplicates of previous tickets, and a supervised classifier answers them in tens of milliseconds at effectively zero marginal cost. Route to the LLM only when the classifier is uncertain.

Stage	Detail
Input	Subject, description, requester department, channel, attachment filenames.
Preprocessing	Lowercase, strip signatures and quoted replies, mask PII (emails, IPs, employee IDs, phone numbers), truncate to a token budget.
Features	Sentence-embedding vector of subject+description, plus structured signals: department one-hot, channel, keyword flags, historical category frequency for the requester.
Fast path model	Logistic regression or LightGBM over the feature vector. Trained on a labelled historical export; class-balanced with weights.
Slow path model	LLM with a structured-output prompt returning strict JSON: category, sub_category, severity, priority, reasoning, confidence. Invoked when fast-path confidence < 0.75 or the predicted class has fewer than N training examples.
Severity logic	Model output combined with deterministic business overrides — e.g. any ticket from a VIP group or containing "production down" is floored at HIGH regardless of model output. Rules always win over the model on the upside.
Priority mapping	Deterministic matrix of severity $\times$ business impact $\times$ affected user count $\rightarrow$ P1-P4. Not a model output; keeps SLA assignment auditable.
Output	Persisted to <code>ticket_classifications</code> with <code>model_version</code> and confidence. Raw model output retained in JSONB.
Feedback loop	Every human override writes a labelled example. A weekly job assembles a retraining set; retraining is a manual, reviewed action — never automatic.
Fallback	If both paths fail: category UNCLASSIFIED, priority P3, routed to the general queue. The system never blocks ticket creation on AI availability.

8.3 Module 2 — RAG Knowledge Retrieval

Ingestion pipeline

- 1. Load** — adapters for Markdown, PDF, DOCX, HTML and Confluence export. Each document keeps source URL, owner and last-modified date.
- 2. Normalise** — strip boilerplate, preserve heading hierarchy, convert tables to a readable text form (tables destroyed by naive extraction are a common silent failure).
- 3. Chunk** — split on headings first, then by token budget (500–800) with 15% overlap. Prepend the heading path to every chunk.
- 4. Embed** — batch through the embedding provider with retry and dimension validation; store in `article_chunks.embedding`.
- 5. Verify** — post-index smoke query per document; record failures to `ingestion_jobs.error_log` rather than failing the whole batch.

Query pipeline

- 1. Query construction** — the Diagnosis agent turns a messy ticket into 2–3 focused search queries. Raw ticket text is a poor query; this step is where most retrieval quality is won.
- 2. Hybrid retrieval** — dense cosine top-30 plus PostgreSQL full-text top-30. Lexical matters here: error codes and product names are exactly what embeddings blur.
- 3. Fusion** — Reciprocal Rank Fusion across both result sets.
- 4. Filter** — drop archived articles, apply category and department access filters.
- 5. Rerank** — cross-encoder scores the fused top-20; keep top-5.
- 6. Pack** — assemble context under a token budget, newest-first on ties, each chunk labelled with its article ID so citations can be resolved.

8.4 Module 3 — Multi-Agent Resolution Framework

Agent	Responsibility	Inputs → Outputs
Diagnosis Agent	Understand the problem; extract symptoms, error codes, affected system, what the user already tried; produce search queries.	Ticket text, classification → problem statement, entities, 2–3 queries
Retrieval Agent	Execute the RAG query pipeline; assess whether retrieved context is sufficient; optionally re-query with reformulated terms.	Queries → ranked chunks, sufficiency verdict
Resolution Agent	Generate ordered troubleshooting steps grounded strictly in retrieved context, with per-step citations.	Problem + context → steps[], summary, citations[]
Validation Agent	Independently check the draft: is every claim supported by a citation? Are steps in a safe order? Any destructive action recommended without warning?	Draft + context → verdict, issues[], adjusted confidence
Escalation Agent	Decide whether to auto-resolve or escalate; select target team using routing rules plus semantic similarity to past escalations; write the handover summary.	Confidence + classification → decision, target team, handover note

Orchestration rules

- Typed shared state.** Agents communicate only through one state object. No agent calls another directly — the orchestrator owns control flow, which is what makes runs reproducible and traceable.
- Conditional edges.** If Retrieval reports insufficient context, the graph loops back to Diagnosis for query reformulation — maximum two loops, then escalate.
- Checkpointing.** State is persisted after each node. A failed run resumes from the last good checkpoint instead of re-burning tokens.
- Per-node limits.** Every node has a timeout, a retry budget (2, with exponential backoff), and a token ceiling. A run that exceeds the total budget terminates and escalates.
- Full tracing.** Every node writes an `agent_steps` row and a Langfuse span. When someone asks "why did it say that?", the answer must be one click away.

Confidence scoring

The final confidence is a weighted composite, not a raw LLM self-report — models are poorly calibrated when asked to score their own certainty. Suggested starting weights, to be tuned against the M2/M3 golden sets:

Signal	Weight	Source
Top-1 retrieval relevance	0.30	Reranker score of the best chunk
Context coverage	0.20	Proportion of generated steps carrying a citation

Validator verdict	0.25	Validation Agent pass/partial/fail
Classification confidence	0.10	Module 1 output
Historical category resolve rate	0.15	Rolling 30-day auto-resolve success for that category

Thresholds (admin-configurable): ≥ 0.85 auto-resolve and send; 0.60–0.85 present as a draft for agent review; < 0.60 escalate immediately without showing a draft, so nobody wastes time on a weak suggestion.

8.5 Guardrails (mandatory, not optional)

Control	Behaviour
PII masking	Emails, phone numbers, IPs, employee IDs and credentials are masked before any text leaves the network boundary; unmasked on display.
Prompt-injection defence	Ticket text is delimited and explicitly labelled as untrusted data. Instruction-like patterns in user content are flagged and never treated as system directives.
Grounding enforcement	Output is rejected if a step has no citation. The model is instructed to state that it cannot answer rather than improvise — an honest "escalating this" beats a confident wrong answer.
Destructive action warning	Steps involving deletion, registry edits, factory resets or credential changes are automatically tagged and require human approval regardless of confidence.
Output schema validation	Every LLM response is parsed against a Pydantic schema; parse failure triggers one repair attempt, then fallback.
Cost control	Per-ticket token ceiling, daily org-wide budget, semantic cache for near-duplicate tickets, automatic downgrade to a cheaper model for simple categories.
Rate & circuit breaking	Provider errors trip a circuit breaker; the system falls back to classifier-only mode and queues generation for retry.

8.6 Evaluation Harness

Prompt and model changes are code changes and must be regression-tested. The evaluation harness runs in CI on every change to `prompts/`, `agents/` or `rag/`.

Golden set	Size	Metrics
Classification set	500 labelled tickets	Accuracy, macro-F1, per-class confusion, severity accuracy
Retrieval set	100 ticket→article pairs	Recall@1/@5, MRR, mean rerank score
Generation set	100 tickets with reference resolutions	Faithfulness, citation coverage, step correctness (LLM-as-judge + spot human review)
Safety set	50 adversarial tickets	Injection resistance, PII leakage, destructive-action tagging

CI gate: a merge is blocked if any headline metric regresses by more than 3% against the current baseline.

9. Third-Party Integrations

Integration	Direction	Mechanism	Implementation Notes
Jira	Bi-directional	REST API + webhooks	Outbound: create issue on escalation with field mapping (priority→Jira priority, category→component, team→assignee group). Inbound: webhook with signature verification updates ticket status. All calls idempotent via a stored external key; exponential backoff on 429; a sync log row per attempt.
Email — inbound	Inbound	IMAP polling (Celery Beat)	Poll every 60s. Parse Message-ID and In-Reply-To to thread replies onto existing tickets instead of creating duplicates. Strip quoted history and signatures before classification. Attachments extracted to object storage with virus scanning.
Email — outbound	Outbound	SMTP / transactional provider	Templates for acknowledgement, resolution, status change, escalation notice, CSAT request. DKIM/SPF configured. Bounce and complaint handling writes back to email_logs. Per-user daily send cap prevents notification storms.
LLM provider	Outbound	HTTPS API	Behind the LLMProvider interface with a secondary provider configured for failover. Streaming supported for interactive drafting.
Embedding provider	Outbound	HTTPS API or local model	Batched. Model version recorded on every chunk — changing models requires a full re-index, and the version field is what makes that detectable.
Object storage	Bi-directional	S3 SDK	Pre-signed URLs only; the API never proxies file bytes. Server-side encryption; lifecycle rules for archival.
SSO / IdP	Inbound	OIDC	Optional. Group claim → role mapping table maintained by Admin.
Slack / Teams	Outbound	Incoming webhook	Optional Phase 1.5: P1 alerts and SLA breach notifications to a team channel.
Sentry	Outbound	SDK	Error tracking across web, Core and AI Service, correlated by request ID.
Langfuse	Outbound	SDK	LLM trace, prompt version, token and cost telemetry.

Integration discipline

Every external call goes through an adapter class with: a timeout, a retry policy, a circuit breaker, structured logging of request/response metadata (never full payloads containing PII), and a health check surfaced on the Integrations page. No vendor SDK is imported outside its adapter module. This is what makes provider swaps a one-file change instead of a refactor.

10. End-to-End Project Workflow

10.1 Operational Flow

Intake — Ticket arrives from portal, email or API. Validated, deduplicated against recent tickets from the same requester, persisted as `NEW`, acknowledgement email sent immediately.

Classification — Async worker calls the classification engine. Category, severity and priority set; SLA due date computed; status → `CLASSIFIED`.

Routing — Routing rules evaluated in order; the ticket is placed in a team queue. High-severity tickets additionally fire an immediate alert.

Investigation — Agent run starts: Diagnosis → Retrieval → Resolution → Validation. Status → `AI_PROCESSING`. Progress streams to the agent workbench.

Decision gate — Composite confidence evaluated against the auto-resolve threshold.

Path A — auto-resolve — Resolution email sent, status → `AI_RESOLVED`, CSAT request scheduled, ticket auto-closes after the confirmation window if the user does not reopen.

Path B — assisted — Draft presented in the agent console. The agent accepts, edits or discards. Status → `IN_PROGRESS`. The edit diff is captured as a quality signal.

Path C — escalate — Escalation record created, Jira issue opened, assignee notified, handover summary attached. Status → `ESCALATED`.

Resolution & closure — Human or AI resolution delivered; requester confirms or reopens; status → `RESOLVED` then `CLOSED`.

Learning — Overrides, edits, reopens and CSAT scores feed the retraining set and the AI quality dashboard. Recurring unresolved patterns generate a "KB gap" report for the Knowledge Manager.

10.2 Ticket State Machine

State	Allowed transitions	Trigger
NEW	→ <code>CLASSIFIED</code> , <code>CANCELLED</code>	Classification completes / requester cancels
CLASSIFIED	→ <code>AI_PROCESSING</code> , <code>ASSIGNED</code>	Agent run starts / manual pickup
AI_PROCESSING	→ <code>AI_RESOLVED</code> , <code>PENDING_REVIEW</code> , <code>ESCALATED</code> , <code>FAILED</code>	Confidence gate outcome
PENDING_REVIEW	→ <code>IN_PROGRESS</code> , <code>ESCALATED</code>	Agent accepts draft or rejects it
ASSIGNED	→ <code>IN_PROGRESS</code> , <code>ESCALATED</code>	Assignee starts work
IN_PROGRESS	→ <code>WAITING_ON_USER</code> , <code>RESOLVED</code> , <code>ESCALATED</code>	Work progresses / info requested
WAITING_ON_USER	→ <code>IN_PROGRESS</code> , <code>CLOSED</code>	User replies / auto-close after timeout
ESCALATED	→ <code>IN_PROGRESS</code> , <code>RESOLVED</code>	Specialist picks up
AI_RESOLVED	→ <code>CLOSED</code> , <code>REOPENED</code>	Confirmation window expires / user reopens
RESOLVED	→ <code>CLOSED</code> , <code>REOPENED</code>	As above
REOPENED	→ <code>IN_PROGRESS</code> , <code>ESCALATED</code>	Re-enters the queue with elevated priority
CLOSED	Terminal	—
FAILED	→ <code>ASSIGNED</code>	AI pipeline error; always falls back to a human

Transitions are enforced centrally in `tickets/state_machine.py`. An illegal transition returns 409 with error code `TICKET_INVALID_TRANSITION`. Every transition writes a `ticket_status_history` row.

11. Development Phases — Step by Step

Week 0 — Mobilisation & Foundations (pre-milestone)

Goal: nobody writes feature code until the skeleton runs end to end.

Track	Tasks
Repository & process	Create three repos (or a monorepo with three packages). Set branch protection, PR template, CODEOWNERS, conventional commits, semantic versioning. Agree the Git branching model (section 14.2).
Environment	Docker Compose bringing up Postgres+pgvector, Redis, Core API, AI Service, web, MinIO, Mailhog. A new developer must reach a running system with one command.
Contracts	Freeze the enum set (status, priority, severity, roles) in one shared document. Publish the initial OpenAPI stub so frontend can start against mocks.
Quality gates	Pre-commit hooks (ruff, black, mypy, eslint, prettier). CI pipeline running lint + test on every PR. Coverage floor set at 60%, raised each milestone.
Data	Obtain and anonymise a historical ticket export. Split into train/validation/hold-out. Curate the initial KB corpus. This is the critical-path item — without labelled data, Milestone 1 cannot be evaluated.
Deliverable	Running skeleton, health endpoints green, CI passing, seeded demo data, published API contract.

Milestone 1 — Weeks 1-2 — Ticket Processing & Classification

Target: classify ≥ 90% of tickets correctly; severity accuracy ≥ 85%.

Track	Tasks
Backend	Accounts app: user model, JWT auth, roles, permission classes. Tickets app: model, state machine, create/list/detail/transition endpoints, filters, cursor pagination. Attachment upload via pre-signed URLs. SLA policy model and due-date computation. Audit middleware. Celery wired with the ingest and ai queues.
AI Service	FastAPI skeleton with health, service auth and structured logging. Preprocessing and PII masking. Feature pipeline. Train the baseline classifier; register model artifacts. LLM provider interface plus one implementation. /v1/classify with the fast/slow path split. Priority mapping matrix. Classification golden set + evaluation script.
Frontend	App shell, layouts, router with role guards. Login and session handling with refresh interceptor. Design tokens and the ui/ primitive set. New Ticket form with Zod validation and attachment upload. My Tickets, Ticket List with server-side filtering, Ticket Detail with the classification panel and confidence meter. Reclassify dialog.
QA	Unit tests on the state machine and permission classes. API contract tests. Playwright journey: login → raise ticket → see classification.
Definition of Done	A ticket raised through the UI is persisted, classified asynchronously, given a severity/priority and SLA date, and appears in a filtered queue. Evaluation report on the hold-out set attached to the milestone PR and both accuracy targets met.

Milestone 2 — Weeks 3-4 — Knowledge Retrieval & Resolution Generation

Target: Recall@5 ≥ 0.85; every generated resolution carries at least one source citation.

Track	Tasks
Backend	Knowledge app: article CRUD, versioning, publish workflow, ingestion job model and status endpoint. Resolutions app: response persistence, citations, accept / edit-send / feedback endpoints. Re-embed-on-publish task. Search proxy endpoint.
AI Service	pgvector schema and HNSW index. Document loaders and heading-aware chunker. Batch embedding with retry. Hybrid retriever with RRF fusion. Cross-encoder reranker. Context packing under a token budget. Resolution prompt v1 with mandatory structured output and citations. Output validator. /v1/retrieve, /v1/resolve, /v1/embed, /v1/index/upsert. Retrieval and generation golden sets wired into CI.
Frontend	Knowledge article list, editor with markdown preview, publish flow, ingestion status page with per-document failures. Resolution panel on ticket detail: step checklist, citation cards linking to source articles, confidence display, accept / edit & send actions. Self-help semantic search page. Live KB suggestions on the ticket creation form.
QA	Retrieval regression suite. Manual review of 100 generated resolutions with a scoring rubric. Load test the retrieve endpoint at expected concurrency.
Definition of Done	KB corpus fully indexed with a verification report. A ticket produces a grounded, cited resolution visible in the UI. Retrieval metrics met and recorded. Editing and sending a resolution works end to end.

Milestone 3 — Weeks 5-6 — Multi-Agent Workflows & Integrations

Target: ≥ 95% agent-run completion; Jira and email delivery ≥ 99% with zero duplicates.

Track	Tasks
Backend	Agents app: run and step models, SSE stream endpoint, retry-from-checkpoint. Escalations app: routing rule engine, escalation records, reassignment, SLA breach sweep task. Jira adapter: client, field mapper, outbound push, inbound webhook with signature

	verification, sync log. Email adapter: IMAP poller with threading, SMTP sender, template engine, bounce handling. Notification dispatcher.
AI Service	Agent base protocol and typed state. Orchestrator graph with conditional edges and checkpointing. Diagnosis, Retrieval, Resolution, Validation, Escalation agents. Composite confidence scorer. Guardrail suite: injection filter, destructive-action tagging, output validation. Langfuse tracing per node. <code>/v1/agents/run</code> , <code>run status</code> , <code>/v1/draft-email</code> . Safety golden set.
Frontend	Agent workbench with live pipeline visualisation. Run detail page showing per-step inputs, outputs, prompt version, tokens and latency. Escalation kanban board with drag-to-reassign. SLA monitor. Integrations page with health cards, Jira config with field mapping UI and test-connection, email config with template editor and variable preview.
QA	Integration suite over 50 complex tickets measuring completion rate. Idempotency tests: replay 200 Jira and email events, assert no duplicates. Chaos test: kill the AI service mid-run, verify graceful degradation and checkpoint recovery.
Definition of Done	A complex ticket flows through all five agents with a full visible trace. Escalation creates a correctly-mapped Jira issue and notifies the assignee. Email replies thread onto the original ticket. All integration reliability targets met.

Milestone 4 — Weeks 7-8 — Dashboard, Optimisation & Deployment

Target: p95 response generation < 5s; CSAT ≥ 85%; production deployment complete.

Track	Tasks
Backend	Analytics app: rollup models, nightly aggregation task, KPI / trends / leaderboard / AI-quality endpoints, CSV and PDF export with async job + download link. Query optimisation: add missing indexes identified by <code>EXPLAIN ANALYZE</code> on the slowest endpoints, eliminate N+1 queries. Redis caching on analytics reads. Audit log partitioning.
AI Service	Semantic response cache. Model routing by category cost tier. Prompt compression and context-budget tuning. Concurrency tuning and connection pooling. Latency profiling to find and remove the p95 tail. Final evaluation run across all four golden sets.
Frontend	Analytics overview, agent performance, AI quality and report builder pages. Admin console: users, roles, teams, SLA policies, routing rules, AI settings, audit log. Accessibility pass (keyboard navigation, ARIA, contrast). Bundle optimisation: route-level code splitting, lazy charts, image and font optimisation. Empty, loading and error states audited on every page.
DevOps	Production Kubernetes manifests / Helm chart. Blue-green or rolling deploy with health gates. Prometheus scraping, Grafana dashboards, alert rules. Automated backups with a tested restore. Secrets migrated to the vault. Load test at 3x expected peak.
QA & UAT	Full regression. Security review: OWASP Top 10 checklist, dependency scan, penetration test of auth and object-level access. UAT with real support staff over 5 days, collecting CSAT. Bug triage and fix cycle.
Documentation	Admin guide, support-agent user guide, runbook (common failures and recovery), API reference published, architecture decision records finalised, handover session recorded.
Definition of Done	System live in production, all evaluation criteria met and evidenced, monitoring and alerting active, backup restore verified, documentation handed over, no open critical or high defects.

12. Deployment Architecture

12.1 Environments

Environment	Purpose	Data	Deploy trigger
Local	Development	Seeded synthetic	Docker Compose, manual
Dev	Shared integration	Synthetic + small KB	Auto on merge to develop
Staging	UAT, load & security testing	Anonymised production-like	Auto on merge to release/*
Production	Live	Real	Manual approval on tagged release

12.2 Runtime Topology (production)

Component	Deployment	Scaling & notes
Web (static SPA)	CDN + object storage	Immutable hashed assets, long cache TTL, index.html no-cache. Zero server cost.
Ingress	Nginx / cloud load balancer	TLS 1.3, HTTP/2, rate limiting, security headers, WAF rules.
Core API	3+ pods (HPA on CPU & RPS)	Gunicorn with Uvicorn workers, readiness and liveness probes, graceful shutdown draining.
AI Service	2+ pods (HPA on queue depth & latency)	Higher memory limits; separate node pool so an AI spike cannot starve the CRUD tier.
Celery workers	Per-queue deployments	ai queue scales on backlog; integrations and notifications run at lower fixed concurrency to respect external rate limits.
Celery Beat	Single replica	Leader-elected; must never run twice or schedules double-fire.
PostgreSQL	Managed instance + read replica	Primary for writes; replica for analytics queries. PITR enabled. pgvector extension on primary.
Redis	Managed, replicated	Separate logical DBs for broker, cache and rate limiting.
Object storage	S3 bucket	Private, SSE enabled, lifecycle transition to cold storage after 180 days.
Observability	Prometheus, Grafana, Loki, Sentry, Langfuse	Dedicated namespace; retention 30 days metrics, 14 days logs, 90 days LLM traces.

12.3 CI/CD Pipeline

1. On pull request — lint (ruff/eslint) → type check (mypy/tsc) → unit tests → coverage gate → dependency & secret scan → build Docker images → container vulnerability scan.
2. AI-specific gate — if prompts/ , agents/ or rag/ changed, run the evaluation harness and block on any headline metric regressing more than 3%.
3. On merge to develop → push images to registry → deploy to Dev → run smoke tests → notify channel.
4. On release branch → deploy to Staging → run full E2E suite and load test → open the UAT window.
5. On tagged release → manual approval → database migration job (backwards-compatible, run before app rollout) → rolling deploy with health gates → automated smoke tests → auto-rollback on failure.

Migration rule Migrations must be backwards-compatible with the previous application version. Adding a non-nullable column without a default, or dropping a column still read by running pods, will cause an outage during a rolling deploy. Column removals happen one release after the code that read them is gone — expand first, contract later.

12.4 Reliability & Recovery

Concern	Policy
Backups	Automated daily full + continuous WAL archiving. Restore drill executed and documented before go-live — an untested backup is not a backup.
RPO / RTO	RPO 15 minutes, RTO 2 hours. Documented in the runbook with the exact recovery sequence.
Degraded modes	LLM provider down → classifier-only mode, all tickets routed to humans, banner shown to agents. Vector store down → keyword-only retrieval. Jira down → queue outbound events for replay.
Alerting	Page on: API 5xx rate > 2%, agent-run failure rate > 10%, Celery queue depth sustained > 500, SLA breach count

	spike, LLM cost exceeding daily budget, database connections > 80%.
Feature flags	Auto-resolve, each agent, and each integration are individually toggleable at runtime. Turning AI off must never break ticketing.

13. Non-Functional Requirements, Testing & Security

13.1 Non-Functional Targets

Attribute	Target	Verification
API latency (CRUD)	p95 < 300 ms	Load test, Prometheus histograms
AI response generation	p95 < 5 s	Measured at the gateway under 20 concurrent users
Classification latency	p95 < 1.5 s	Fast path should land under 200 ms
Throughput	500 tickets/hour sustained	Load test at 3× expected peak
Availability	99.5% monthly (core ticketing)	Uptime monitoring; AI degradation does not count as downtime
Frontend load	LCP < 2.5 s, bundle < 300 KB gzipped initial	Lighthouse in CI
Accessibility	WCAG 2.1 AA	axe automated scan + manual keyboard pass
Data retention	Tickets 3 years, audit 7 years, LLM traces 90 days	Retention job with documented policy

13.2 Testing Strategy

Level	Tooling	Coverage expectation
Unit	pytest, Vitest	≥ 80% on services, selectors, state machine, guardrails, hooks and utilities.
Component	React Testing Library	Every shared UI component and every form.
API contract	pytest + schemathesis	Every endpoint validated against its OpenAPI schema, including error shapes.
Integration	pytest + testcontainers	Real Postgres and Redis. External APIs mocked at the adapter boundary with recorded fixtures.
AI evaluation	Custom harness	Four golden sets, run in CI on every AI-layer change.
E2E	Playwright	Eight critical journeys covering each role.
Load	k6 / Locust	Sustained and spike profiles before each release.
Security	OWASP ZAP, bandit, npm audit, Trivy	Automated in CI; manual penetration test before go-live.

13.3 Security Controls

- **Transport:** TLS everywhere including service-to-service. HSTS, CSP, X-Frame-Options, X-Content-Type-Options headers set at the edge.
- **Input:** Every payload validated by serializer or Pydantic model. Parameterised queries only. Uploaded files checked by magic-byte type detection, size cap and virus scan before they are readable.
- **Output:** Markdown rendered through a sanitiser. No `dangerouslySetInnerHTML` on any AI-generated or user-supplied content without sanitisation.
- **Access:** Object-level checks in service functions. Role-scoped querysets. IDOR tests in the security suite for every resource-by-id endpoint.
- **Secrets:** Vault-managed, injected at runtime, rotated quarterly. CI fails on any detected secret in a diff.
- **AI-specific:** PII masking before external calls, prompt-injection filtering, output schema validation, provider data-retention set to zero where the vendor supports it, and no customer data used for vendor training.
- **Audit:** Append-only log of every privileged action, immutable at the database level, with actor, request ID, before and after state.

14. Team Structure, Ways of Working & Risk Register

14.1 Suggested Squad Split

Squad	Size	Ownership
Platform & Core API	3-4	Auth, tickets, state machine, SLA, escalation, audit, database migrations.
AI / ML	3-4	Classification, RAG, agents, guardrails, evaluation harness, prompt versioning.
Frontend	3-4	All pages, design system, state, accessibility, performance.
Integrations	2	Jira, email in/out, notifications, webhooks, adapter reliability.
Data & Analytics	2	Rollups, dashboards, exports, KB ingestion pipeline, dataset curation.
QA & DevOps	2	CI/CD, environments, test automation, monitoring, security scanning, release management.

Each squad nominates a rotating reviewer. Cross-squad contracts (API schemas, enums, event shapes) are agreed in writing before implementation starts — the most expensive delays on projects of this shape come from two squads building against different assumptions about the same field.

14.2 Ways of Working

Practice	Standard
Branching	main (production) ← release/* ← develop ← feature/SP-123-short-slug. Hotfixes branch from main and merge back to both.
Commits	Conventional commits (feat:, fix:, chore:) — drives automated changelog generation.
Pull requests	Maximum ~400 changed lines, two approvals, CI green, self-review checklist completed. No direct pushes to protected branches.
Sprints	One-week sprints inside each two-week milestone. Demo every Friday against the milestone's Definition of Done, not against "work completed".
Ceremonies	15-minute daily standup, mid-milestone technical design review, end-of-milestone retrospective with actions carried into the next.
Documentation	Architecture Decision Record for every significant choice (why this vector store, why this threshold). README kept runnable. API contract regenerated on every merge.
Definition of Ready	A ticket enters a sprint only with acceptance criteria, an API contract if it crosses a service boundary, and a test approach noted.

14.3 Risk Register

Risk	Impact	Likelihood	Mitigation
Insufficient or poorly labelled historical ticket data	High	High	Start data acquisition in Week 0. If labels are thin, begin with an LLM-only classification path and collect labels from agent overrides through Milestone 1, training the supervised model in Milestone 2 instead.
Sparse or outdated knowledge base → weak retrieval	High	High	Audit KB coverage against the top 20 ticket categories in Week 1. Produce a gap report and have the Knowledge Manager author missing articles in parallel with development. Retrieval quality is capped by corpus quality — no amount of model tuning compensates.
LLM cost overrun	Medium	Medium	Fast-path classifier avoids most LLM calls. Semantic caching, token ceilings, per-category model routing, daily budget alerts, and cost visible on the AI quality dashboard from Milestone 2.
Latency target missed under load	Medium	Medium	Asynchronous processing means the user never waits. Measure p95 from Milestone 2, not Milestone 4. Parallelise independent agent nodes; cache retrieval for duplicate tickets.
Hallucinated resolutions damaging trust	High	Medium	Mandatory citations, validation agent, conservative auto-resolve threshold at launch (start at 0.90 and lower it only with evidence), destructive-action gating, and visible feedback capture.
Jira or mailbox API changes / rate limits	Medium	Low	Adapter isolation, contract tests against recorded fixtures, exponential backoff, dead-letter queue with replay.
Scope creep across eight weeks	High	Medium	Milestone Definition of Done is contractual. New requests go to a Phase 2 backlog by default; only the project sponsor can promote one, and only by removing something of equal size.
Support staff resist AI suggestions	Medium	Medium	Position the tool as assistive from day one. Involve two support agents in UAT from Milestone 2. Make every suggestion editable and every trace inspectable — trust follows transparency.

14.4 Post-Launch Roadmap (indicative)

- **Phase 2:** Conversational chat widget with live clarifying questions; Slack/Teams ticket raising; proactive KB gap detection with auto-drafted article suggestions.
- **Phase 3:** Guarded automated remediation (password reset, group membership, license assignment) behind an approval workflow; predictive SLA breach warnings.
- **Phase 4:** Fine-tuned domain classifier; multilingual support; multi-tenant isolation for managed-service delivery.